Secure Pharmacy Inventory and Prescription Management System with Medicine Stock Tracking

A CAPSTONE PROJECT REPORT

Submitted in the partial fulfilment for the Course of

CSA0211 – C Programming

to the award of the degree of

BACHELOR OF ENGINEERING
IN
ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

Kaviyaa Sri M (192312614)
Hema Sai Lahari Y (192311446)
Manasa G (192525315)

Under the Supervision of
Dr. S.K. Saravanan

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

August 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-602105

DECLARATION

We, Kaviyaasri M from ECE, Lahari from CSE, Manasa from CSE, Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled "Design and Development of a Smart Kitchen Safety System Using MQ-2 Chemical Sensor for Real-Time Gas and Smoke Detection" is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: Chennai, Thandalam Campus

Date:

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "**Secure Pharmacy Inventory and Prescription Management System with Medicine Stock Tracking**" has been carried out by Kaviyaa Sri M, Lahari, Manasa, under the supervision of **Dr. S.K. Saravanan** is submitted in partial fulfilment of the requirements for the current semester of the B.E ECE program at Saveetha Institute of Medical and Technical Sciences, Chennai.

SIGNATURE	**SIGNATURE**
Program Director	**Dr. S.K. Saravanan**
Department Of CSE	Professor
Saveetha School of Engineering SIMATS	Saveetha School of Engineering SIMATS

Submitted for the Project work Viva-Voce held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. S. Suresh Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh, for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department and our Program Director of CSE for their continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, Dr. S.K. Saravanan for his creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members, and the entire faculty team for their constructive feedback. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

ABSTRACT

The Secure Pharmacy Inventory and Prescription Management System with Medicine Stock Tracking is a computerized pharmacy management solution designed to efficiently manage medicine inventory, patient prescriptions, and medicine stock information. The system aims to reduce the difficulties associated with manual pharmacy management by providing an organized and reliable method for maintaining medicine records, patient details, prescription information, stock quantities, and expiry dates. The system improves the overall efficiency of pharmacy operations by reducing manual effort, paperwork, and human errors.

The proposed system consists of three major modules: Medicine Inventory Management, Prescription and Patient Medicine Management, and Stock and Expiry Monitoring. The Medicine Inventory Management module allows users to add, update, search, and delete medicine records while maintaining information about medicine availability. The Prescription and Patient Medicine Management module manages patient details, generates prescriptions, assigns medicines, and maintains prescription records. The Stock and Expiry Monitoring module monitors medicine quantities, identifies low-stock medicines, tracks expiry dates, and supports effective stock management. The system is implemented using C programming concepts such as Structures, Arrays, Strings, and File Handling for organized and persistent data storage. Algorithms and operations such as Linear Search, CRUD operations, Stock Update, Expiry Check, File Handling, and Data Validation are used to perform the required functions efficiently.

The proposed system provides advantages such as improved data accuracy, organized record management, reduced manual errors, faster operations, and efficient monitoring of medicine stock and expiry dates. It reduces the need for paper-based records and manual stock updates while providing a secure and reliable approach to managing pharmacy information. The system can be useful for pharmacies and medical stores where maintaining accurate inventory and prescription records is essential for efficient operations. By integrating inventory management, prescription handling, and stock monitoring into a single system, the proposed solution provides a practical approach to improving the reliability, accuracy, and efficiency of pharmacy management.

TABLE OF CONTENTS

CHAPTER 1
INTRODUCTION

1.1 Background Information

The increasing need for efficient and accurate pharmacy management has created challenges in maintaining medicine inventory, patient prescriptions, stock quantities, and expiry information. Traditional pharmacy management methods often depend on manual record maintenance and paper-based prescriptions, which can lead to human errors, delays, difficulty in tracking stock, and improper monitoring of medicine expiry dates. To improve pharmacy operations and reduce manual effort, computerized management systems can be used to organize pharmacy records and provide reliable inventory and prescription management.

The Secure Pharmacy Inventory and Prescription Management System with Medicine Stock Tracking is developed to provide an organized solution for managing pharmacy operations. The system maintains medicine records, patient details, prescriptions, stock quantities, and expiry information. It is implemented using C programming concepts such as Structures, Arrays, Strings, and File Handling for storing and managing data efficiently. The system helps reduce manual errors, improve data accuracy, and provide faster and more reliable pharmacy management.

1.2 Project Objectives

The main objectives of this project are:

• To develop a secure system for managing pharmacy inventory and patient prescriptions.

• To maintain accurate records of medicines, patients, prescriptions, and stock quantities.

• To monitor medicine stock levels and expiry dates effectively.

• To reduce manual errors, paperwork, and time required for pharmacy operations.

• To improve data organization, accuracy, and efficiency in pharmacy management.

1.3 Significance of the Project

The Secure Pharmacy Inventory and Prescription Management System with Medicine Stock Tracking plays an important role in improving the efficiency and reliability of pharmacy operations. Manual maintenance of medicine and prescription records can result in data loss, incorrect stock information, delayed processing, and difficulty in identifying expired medicines. Proper management of these records is essential for ensuring accurate inventory control and efficient handling of patient prescriptions. This project provides a computerized solution for maintaining and organizing pharmacy-related information.

The project mainly focuses on inventory management, prescription management, and stock and expiry monitoring. By maintaining medicine and patient records systematically, the

system reduces dependence on manual record keeping and improves data accuracy. Operations such as searching, adding, updating, and deleting records help users manage pharmacy information efficiently. The proposed system provides a simple, secure, and reliable approach for improving pharmacy operations and reducing the time and effort required for manual management.

1.4 Scope of the Project

The scope of this project includes:

• The project focuses on managing medicine inventory and patient prescription records.

• It is designed to maintain medicine details, stock quantities, availability, and expiry information.

• The system supports adding, updating, searching, and deleting medicine and prescription records.

• It enables monitoring of medicine stock levels and identification of expired or low-stock medicines.

• The system is intended to reduce manual effort and improve the accuracy and efficiency of pharmacy management.

1.5 Methodology Overview

The methodology of the Secure Pharmacy Inventory and Prescription Management System with Medicine Stock Tracking begins with collecting and storing medicine, patient, and prescription information. The system uses Structures, Arrays, Strings, and File Handling in C to organize and maintain the required records. Users can perform operations such as adding, updating, searching, and deleting records, while the system validates the entered information to maintain data accuracy.

The system also monitors medicine quantities and expiry dates to support effective stock management. Stock quantities are updated based on medicine records, while the expiry checking process identifies medicines that have reached their expiry date. File handling is used to store and retrieve information for future operations. These processes enable the system to provide organized inventory management, efficient prescription handling, and reliable medicine stock tracking while reducing manual errors and improving overall pharmacy efficiency.

CHAPTER 2
PROBLEM IDENTIFICATION AND ANALYSIS

2.1 Description of the Problem

Manual management of pharmacy inventory and patient prescriptions can lead to several difficulties in maintaining accurate and organized records. Medicines need to be monitored regularly for stock availability, quantity, and expiry dates, while patient prescriptions and medicine details must also be maintained properly. Traditional paper-based record keeping can result in human errors, data duplication, loss of records, and delays in updating information. These problems can affect the efficiency of pharmacy operations and make it difficult to manage medicine and prescription information accurately.

Another major problem is the lack of effective stock and expiry monitoring in manual pharmacy management. Pharmacy staff may find it difficult to identify low-stock medicines or expired medicines when records are maintained manually. Similarly, managing patient details and prescription records separately can increase the time required for retrieving and updating information. Therefore, there is a need for a computerized pharmacy management system that can organize inventory, prescription, and stock information efficiently. Such a system can reduce manual effort, improve accuracy, and support faster and more reliable pharmacy operations.

2.2 Evidence of the Problem

Manual maintenance of medicine records, stock information, and prescriptions can increase the possibility of incorrect entries and delayed updates. Medicines with limited stock may not be identified on time, while expired medicines can remain unnoticed when expiry information is not regularly monitored. Paper-based prescription records can also make it difficult to retrieve patient and medicine information quickly. These issues can affect inventory control and create additional workload for pharmacy staff.

Existing pharmacy management practices that depend mainly on manual record maintenance may not provide an organized method for searching, updating, and storing information. As the number of medicines, patients, and prescriptions increases, managing records manually becomes more time-consuming and difficult. Therefore, there is a need for a computerized system that can provide efficient inventory and prescription management. Implementing functions such as CRUD operations, stock updates, expiry checking, file handling, and data validation can provide a reliable solution for improving pharmacy management and reducing manual errors.

2.3 Stakeholders

The primary stakeholders of this project are pharmacists and pharmacy staff, as they directly use the system to manage medicine inventory, patient information, prescriptions, stock quantities, and expiry details. The system helps them add, update, search, and delete records efficiently while reducing the time and effort required for manual record maintenance. It also supports better monitoring of medicine availability and expiry information.

Patients are also important stakeholders because organized prescription and medicine records can help improve the accuracy and efficiency of prescription management. Pharmacy owners and management teams can benefit from improved inventory control, reduced manual errors, and better organization of pharmacy information. The system can support efficient daily operations and provide a more reliable method for managing pharmacy records.

Students, researchers, and developers working in the fields of C programming, database-oriented applications, and pharmacy management systems are additional stakeholders of the project. They can use the system as a foundation for developing more advanced pharmacy management applications. Educational institutions may also use the project for academic learning, practical programming exercises, and demonstrations of Structures, Arrays, Strings, File Handling, and data management concepts.

2.4 Supporting Data and Research

Research and practical observations of pharmacy operations indicate that maintaining accurate medicine inventory and prescription records is essential for efficient pharmacy management. Manual record keeping can result in incorrect stock information, difficulty in tracking expiry dates, loss of records, and delays in retrieving patient or medicine details. As the number of medicines and prescriptions increases, maintaining these records manually becomes more difficult and time-consuming. Therefore, there is a growing need for computerized systems that can organize pharmacy information and reduce dependence on manual processes.

The use of computer-based inventory and record management systems provides an effective approach for improving data organization and operational efficiency. Programming concepts such as Structures, Arrays, Strings, and File Handling can be used to store, manage, and retrieve pharmacy information systematically. Functions such as Linear Search, CRUD operations, Stock Update, Expiry Check, and Data Validation further support efficient record management. By integrating these functions into a single system, the proposed solution improves medicine inventory management, prescription handling, stock monitoring, and expiry tracking while reducing manual errors and improving the overall reliability of pharmacy operations.

CHAPTER 3

SOLUTION DESIGN AND IMPLEMENTATION

3.1 Development and Design Process

The development and design process of the Secure Pharmacy Inventory and Prescription Management System with Medicine Stock Tracking began with analyzing the common difficulties involved in manual pharmacy management. The system was designed to organize medicine inventory, patient details, prescription records, stock quantities, and expiry information efficiently. The project uses C programming concepts such as Structures, Arrays, Strings, and File Handling to maintain pharmacy records and reduce errors associated with manual record management.

After identifying the required functions, the system was divided into three major modules: Medicine Inventory Management, Prescription and Patient Medicine Management, and Stock and Expiry Monitoring. The system allows users to add, update, search, and delete records while maintaining medicine availability and prescription information. Stock quantities are updated during inventory operations, while expiry checking helps identify expired medicines. This design provides an organized and reliable approach for managing pharmacy information and improving overall operational efficiency.

3.2 Tools and Technologies Used

The Secure Pharmacy Inventory and Prescription Management System is implemented using the C programming language and fundamental data management concepts. Structures are used to organize medicine, patient, and prescription information, while Arrays and Strings are used for storing and processing multiple records and textual data. File Handling is used to store and retrieve pharmacy information so that records can be maintained for future operations.

The system also uses programming operations and algorithms such as Linear Search, CRUD operations, Stock Update, Expiry Check, and Data Validation. These functions support efficient searching, modification, deletion, validation, and monitoring of pharmacy records. The combination of these programming concepts provides a simple and organized solution for pharmacy inventory and prescription management.

3.3 Solution Overview

The Secure Pharmacy Inventory and Prescription Management System with Medicine Stock Tracking is a computerized solution developed to overcome the limitations of manual pharmacy management. The system provides an organized method for maintaining medicine inventory, patient information, prescription records, stock quantities, and expiry details. It reduces manual paperwork and helps improve the accuracy and efficiency of daily pharmacy operations.

The system is developed using the C programming language and incorporates Structures, Arrays, Strings, and File Handling for effective data management. It consists of three major

modules: Medicine Inventory Management, Prescription and Patient Medicine Management, and Stock and Expiry Monitoring. The system supports operations such as adding, updating, searching, and deleting records, while also providing stock updates and expiry checking. This integrated approach provides a simple and reliable solution for managing pharmacy information.

3.4 Engineering Standards Applied

- Standard C programming practices are followed to ensure structured, readable, and reliable program development.
- Proper data organization techniques are applied using Structures, Arrays, and Strings for maintaining pharmacy records.
- File handling principles are used for systematic storage and retrieval of medicine, patient, and prescription information.
- Data validation techniques are implemented to minimize incorrect and invalid user inputs.
- Systematic inventory management practices are followed for maintaining accurate medicine stock and expiry information.

3.5 Solution Justification

The proposed system is justified by the need to reduce the difficulties associated with manual pharmacy management. Manual records require more time and effort and can result in errors, misplaced information, and difficulty in tracking medicine stock and expiry dates. The proposed computerized system addresses these problems by providing organized record management and efficient inventory monitoring.

The use of C programming concepts such as Structures, Arrays, Strings, and File Handling makes the system suitable for managing pharmacy data in a simple and structured manner. The integration of inventory management, prescription handling, stock updating, and expiry checking provides a practical solution for improving pharmacy operations. Therefore, the proposed system helps reduce manual errors, improve data accuracy, save processing time, and provide a more efficient and reliable method for pharmacy management.

CHAPTER 4

RESULTS AND RECOMMENDATIONS

4.1 Evaluation of Results

The Secure Pharmacy Inventory and Prescription Management System with Medicine Stock Tracking was successfully developed and tested for managing pharmacy inventory, patient prescriptions, medicine stock, and expiry information. The system effectively performed operations such as adding, updating, searching, and deleting medicine records while maintaining organized patient and prescription information. During testing, the system successfully updated medicine stock quantities, stored and retrieved records using file handling, and identified medicines based on their expiry information.

The results showed that the inventory and prescription management processes worked efficiently with improved accuracy and reduced manual effort. The system provided reliable record management through Structures, Arrays, Strings, and File Handling, while data validation helped reduce incorrect user inputs. The stock monitoring and expiry checking functions supported better inventory control and helped identify medicines requiring attention. Overall, the developed system provided a simple, secure, and reliable solution for improving pharmacy operations and reducing manual errors.

4.2 Challenges Encountered

• Managing multiple medicine, patient, and prescription records efficiently.

• Maintaining accurate medicine stock quantities during stock updates.

• Implementing proper expiry checking for medicine records.

• Handling file storage and retrieval without losing existing pharmacy information.

• Validating user inputs to prevent incorrect or invalid data entries.

• Ensuring correct searching, updating, and deletion of records during different operations.

4.3 Possible Improvements

• Developing a graphical user interface for easier and more user-friendly pharmacy management.

• Integrating a database system for handling larger volumes of medicine, patient, and prescription records.

• Adding automatic notifications for low-stock and approaching-expiry medicines.

• Implementing user authentication and role-based access for improved system security.

• Adding report generation features for inventory, prescriptions, stock levels, and expired medicines.

• Developing a web or mobile-based version for remote access and improved accessibility.

4.4 Recommendations

The Secure Pharmacy Inventory and Prescription Management System with Medicine Stock Tracking can be further improved and implemented in pharmacies and medical stores to support efficient inventory and prescription management. Regular updating of medicine records, stock quantities, and expiry information is recommended to maintain accurate inventory data. Proper data validation and backup procedures should also be followed to ensure reliable storage and retrieval of pharmacy information.

Additional features such as database integration, automatic low-stock and expiry alerts, user authentication, and report generation can further improve the effectiveness of the system. Regular testing and maintenance are recommended to ensure reliable operation when the number of medicines, patients, and prescriptions increases. By implementing these improvements, the system can provide a more comprehensive, secure, and efficient solution for modern pharmacy management.

CHAPTER 5
REFLECTION OF LEARNING AND PERSONAL DEVELOPMENT

5.1 Key Learning Outcomes

Through the development of the Secure Pharmacy Inventory and Prescription Management System with Medicine Stock Tracking, valuable knowledge and practical experience were gained in the areas of C programming, data management, file handling, and software development. The project helped in understanding how Structures, Arrays, Strings, and Files can be used to organize and manage pharmacy-related information. It also improved problem-solving abilities, logical thinking, program design skills, debugging techniques, and data validation practices. In addition, the project provided practical experience in developing a real-world application for managing medicine inventory, patient prescriptions, stock quantities, and expiry information.

5.2 Technical Skills

• C programming and structured programming

• Structures, Arrays, and Strings implementation

• File handling for data storage and retrieval

• CRUD operations and Linear Search implementation

• Medicine stock and expiry management

• Data validation and error handling

• Program testing and debugging

5.3 Problem-Solving and Critical Thinking

The development of the Secure Pharmacy Inventory and Prescription Management System improved problem-solving and critical thinking abilities by providing practical experience in handling different software and data management challenges. During the project, issues related to record storage, stock updates, searching, data validation, and expiry checking were identified and resolved through testing and logical analysis. The project also required decision-making for organizing the system into different modules and selecting suitable programming concepts for managing pharmacy records. Debugging program errors and

ensuring accurate data processing enhanced analytical thinking and helped in developing effective solutions for reliable pharmacy management.

5.4 Applications of Engineering Standards

Engineering standards and systematic programming practices were applied throughout the project to ensure reliable program operation, organized data management, and accurate processing of pharmacy records. Structured programming principles, proper data organization techniques, file handling methods, and input validation practices were followed during development. Systematic approaches were used for implementing inventory operations, prescription management, stock updates, and expiry checking. These practices helped improve program reliability, reduce data-related errors, and maintain efficient performance throughout the pharmacy management system.

5.5 Insights into the Industry

The project provided useful insights into the fields of software development, data management, healthcare applications, and computerized pharmacy management. It demonstrated how software systems can be used to reduce manual work, improve data accuracy, and efficiently manage large amounts of information in real-world environments. The development process also highlighted the importance of organized data storage, reliable record management, automation, and user input validation in healthcare-related applications. Additionally, the project showed how programming concepts and software solutions can be applied to develop practical systems that improve operational efficiency and support better management of pharmacy services.

CHAPTER 6

CONCLUSION

The Secure Pharmacy Inventory and Prescription Management System with Medicine Stock Tracking was successfully developed as an effective solution for managing pharmacy inventory, patient prescriptions, medicine stock, and expiry information. The project demonstrated how C programming concepts such as Structures, Arrays, Strings, and File Handling can be integrated to provide organized and reliable pharmacy data management. The system was capable of performing important operations such as adding, updating, searching, and deleting records while maintaining medicine availability and prescription information, thereby improving the efficiency of pharmacy operations.

The project also highlighted the advantages of computerized pharmacy management systems in reducing manual supervision, paperwork, and human errors. Through the use of structured data management, CRUD operations, Linear Search, stock updating, expiry checking, and data validation, efficient management of pharmacy records was achieved using simple programming techniques. Overall, the project provides a practical, organized, and reliable solution for improving medicine inventory management, prescription handling, and stock monitoring.

The development of this system also emphasized the importance of applying software technology to improve efficiency and accuracy in pharmacy operations. By integrating inventory management, prescription records, stock monitoring, and expiry checking into a single system, the project demonstrated how computerized solutions can simplify real-world management tasks. The knowledge and experience gained through this project can be applied to various healthcare and inventory management applications in the future. With further enhancements such as database integration, user authentication, automatic alerts, report generation, and web or mobile connectivity, the system can become a more comprehensive and secure pharmacy management solution.

CHAPTER 7
REFERENCES

- Devnani, M., Gupta, A. K., & Nigah, R. (2010). *ABC and VED Analysis of the Pharmacy Store of a Tertiary Care Teaching, Research and Referral Healthcare Institute of India*. **Journal of Young Pharmacists, 2(2), 201–205**.

- *Improving medical stores management through automation and effective communication*. (2015). **Medical Journal Armed Forces India**. This study specifically examined the problems of **manual medical-store inventory management** and developed an inventory management system that included stock and expiry monitoring.

- Meena, D. K., & Mathaiyan, J. (2024). *Unveiling supply chain efficiency: exploring ABC-VED analysis studies on drug inventory management in India*. **International Journal of Basic & Clinical Pharmacology, 13(4), 563–567**. https://doi.org/10.18203/2319-2003.ijbcp20241660

- *Assessment of drug inventory using ABC-VED matrix analysis in selected public health facilities of Puducherry, India*. (2025). **Journal of Family Medicine and Primary Care**. https://doi.org/10.4103/jfmpc.jfmpc_1544_24

- Jain, K., & Makkar, N. (2026). *ABC–VED matrix analysis of drug inventory management in a tertiary care teaching hospital: a retrospective observational study*. **International Journal of Basic & Clinical Pharmacology, 15(3), 479–485**. https://doi.org/10.18203/2319-2003.ijbcp20261112

- Ilango, G., Nishanthi, A., Navabalan, V., Ananthy, V., Manickam, S., & Suryakumar, C. (2026). *Drug Inventory Management in a Pharmacy Store of a Tertiary Care Hospital Using Always Better Control (ABC) and Vital, Essential, and Desirable (VED) Analysis*. **Cureus, 18(6), e110514**. https://doi.org/10.7759/cureus.110514

- Sinha, B., et al. (2025). *Evaluation and Optimization of Pharmaceutical Inventory Management in a Tertiary Care Teaching Hospital in Jharkhand, India, Using ABC-VED Analysis*.

- Ahmed, S. K., Naji, Z. H., Hatif, Y. N., & Hussam, M. (2020). *Design and Implementation of a Computerized Drug Inventory Management Information System Using ASP.NET MVC*. **Diyala Journal of Engineering Sciences, 13(4)**. https://doi.org/10.24237/djes.2020.13410.

- Kertapradja, E. N., & Basry, A. (2026). *Membangun Sistem Inventory Stok Obat Berbasis Web dengan Metode Min-Max Stock Level pada Apotek Mulia Jakarta*. **IKRA-ITH Informatika: Jurnal Komputer dan Informatika, 10(1), 49–57**. https://doi.org/10.37817/ikraith-informatika.v10i1.5702.

CHAPTER 8
APPENDICES

Code

```
/*
================================================================
==================

 SECURE PHARMACY INVENTORY AND PRESCRIPTION MANAGEMENT SYSTEM
 WITH MEDICINE STOCK TRACKING

================================================================
==================

 Concepts demonstrated:
   - Structures (Medicine, Prescription, PrescribedItem, Admin)
   - Arrays of structures (inventory[], prescriptions[])
   - Functions (modular design, 25+ functions)
   - File Handling (binary files for persistent storage)
   - String handling (string.h)
   - Login/Authentication ("Secure" system access)
   - Searching (linear search by ID / name)
   - Sorting (bubble sort by name / quantity)
   - Enumerations (Category)
   - Pointers (passed to functions by reference)
   - Control structures (switch-case menu, loops)

================================================================
==================

 Compile : gcc pharmacy_management.c -o pharmacy
 Run     : ./pharmacy
 Default login -> Username: admin   Password: admin123

================================================================
==================
```

```
*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

/* ---------------------------- CONSTANTS ---------------------------------- */
#define MAX_MEDICINES      200
#define MAX_PRESCRIPTIONS  200
#define MAX_ITEMS_PER_PRESCRIPTION 10
#define MEDICINE_FILE      "medicines.dat"
#define PRESCRIPTION_FILE  "prescriptions.dat"
#define LOW_STOCK_THRESHOLD 10
#define ADMIN_USER "admin"
#define ADMIN_PASS "admin123"

/* ---------------------------- ENUMS -------------------------------------- */
typedef enum {
    TABLET,
    SYRUP,
    INJECTION,
    CAPSULE,
    OINTMENT,
    DROPS,
    OTHER
} Category;
```

```
const char *categoryNames[] = {
    "Tablet", "Syrup", "Injection", "Capsule", "Ointment", "Drops", "Other"
};
/* ---------------------------- STRUCTURES ---------------------------------- */
typedef struct {
    int    id;
    char   name[50];
    char   manufacturer[50];
    char   batchNo[20];
    Category category;
    int    quantity;
    float  unitPrice;
    char   mfgDate[12];     /* DD-MM-YYYY */
    char   expiryDate[12];  /* DD-MM-YYYY */
    int    isActive;        /* 1 = active, 0 = deleted */
} Medicine;
typedef struct {
    int   medicineId;
    char  medicineName[50];
    int   quantity;
    float subTotal;
} PrescribedItem;
typedef struct {
    int    prescriptionId;
    char   patientName[50];
    int    patientAge;
    char   doctorName[50];
```

```
    char   date[12];
    PrescribedItem items[MAX_ITEMS_PER_PRESCRIPTION];
    int    itemCount;
    float  totalAmount;
} Prescription;
/* ---------------------------- GLOBAL ARRAYS ---------------------------------- */
Medicine     inventory[MAX_MEDICINES];
Prescription prescriptions[MAX_PRESCRIPTIONS];
int medicineCount = 0;
int prescriptionCount = 0;
int nextMedicineId = 1;
int nextPrescriptionId = 1001;
/* ---------------------------- FUNCTION PROTOTYPES ---------------------------- */
int  loginScreen();
void mainMenu();
/* Inventory operations */
void addMedicine();
void displayAllMedicines();
void searchMedicineMenu();
int  findMedicineIndexById(int id);
int  findMedicineIndexByName(const char name[]);
void updateMedicineStock();
void deleteMedicine();
void checkLowStock();
void checkExpiringMedicines();
void sortMedicinesByName();
void sortMedicinesByQuantity();
```

```
void printMedicineRow(Medicine m);
/* Prescription operations */
void createPrescription();
void viewAllPrescriptions();
void viewPrescriptionById();
void printPrescription(Prescription p);
/* File handling */
void saveMedicinesToFile();
void loadMedicinesFromFile();
void savePrescriptionsToFile();
void loadPrescriptionsFromFile();
/* Utility */
void clearInputBuffer();
void toLowerStr(char *str);
void pause_screen();


/*
================================================================
==========
  MAIN FUNCTION
================================================================
========== */
int main() {
    loadMedicinesFromFile();
    loadPrescriptionsFromFile();
    printf("=====================================================\n");
    printf("  SECURE PHARMACY INVENTORY & PRESCRIPTION SYSTEM\n");
    printf("=====================================================\n");
    if (!loginScreen()) {
```

```
        printf("\nToo many failed login attempts. Exiting for security.\n");
        return 0;
    }
    mainMenu();
    /* Persist data before exit */
    saveMedicinesToFile();
    savePrescriptionsToFile();
    printf("\nAll data saved securely. Goodbye!\n");
    return 0;
}
/*
================================================================
==========
   LOGIN / SECURITY
================================================================
========== */
int loginScreen() {
    char user[30], pass[30];
    int attempts = 3;
    while (attempts > 0) {
        printf("\n--- ADMIN LOGIN ---\n");
        printf("Username: ");
        scanf("%29s", user);
        printf("Password: ");
        scanf("%29s", pass);
        if (strcmp(user, ADMIN_USER) == 0 && strcmp(pass, ADMIN_PASS) == 0) {
            printf("\nLogin successful! Welcome, %s.\n", user);
            return 1;
```

```
        } else {
            attempts--;
            printf("Invalid credentials. Attempts left: %d\n", attempts);
        }
    }
    return 0;
}
/*
================================================================
==========
  MAIN MENU
================================================================
========== */
void mainMenu() {
    int choice;
    do {
printf("\n=====================================================\n");
        printf("                  MAIN MENU\n");
        printf("=====================================================\n");
        printf(" 1. Add New Medicine\n");
        printf(" 2. Display All Medicines\n");
        printf(" 3. Search Medicine (by ID / Name)\n");
        printf(" 4. Update Medicine Stock\n");
        printf(" 5. Delete Medicine\n");
        printf(" 6. Check Low Stock Alert\n");
        printf(" 7. Check Expiring Medicines\n");
        printf(" 8. Sort Medicines by Name\n");
        printf(" 9. Sort Medicines by Quantity\n");
        printf("10. Create New Prescription (Dispense Medicine)\n");
```

```
 printf("11. View All Prescriptions\n");
 printf("12. View Prescription by ID\n");
 printf("13. Save Data Now\n");
 printf(" 0. Exit\n");
printf("==================================================\n");
 printf("Enter your choice: ");
 if (scanf("%d", &choice) != 1) {
    clearInputBuffer();
    choice = -1;
 }
 switch (choice) {
    case 1:  addMedicine(); break;
    case 2:  displayAllMedicines(); break;
    case 3:  searchMedicineMenu(); break;
    case 4:  updateMedicineStock(); break;
    case 5:  deleteMedicine(); break;
    case 6:  checkLowStock(); break;
    case 7:  checkExpiringMedicines(); break;
    case 8:  sortMedicinesByName(); break;
    case 9:  sortMedicinesByQuantity(); break;
    case 10: createPrescription(); break;
    case 11: viewAllPrescriptions(); break;
    case 12: viewPrescriptionById(); break;
    case 13:
        saveMedicinesToFile();
        savePrescriptionsToFile();
        printf("\nData saved successfully.\n");
```

```
            break;
        case 0:  printf("\nExiting system...\n"); break;
        default: printf("\nInvalid choice. Please try again.\n");
      }
   } while (choice != 0);
}
/*
================================================================
==========
  INVENTORY: ADD MEDICINE
================================================================
========== */
void addMedicine() {
   if (medicineCount >= MAX_MEDICINES) {
      printf("\nInventory full! Cannot add more medicines.\n");
      return;
   }
   Medicine m;
   int catChoice;
   clearInputBuffer();
   m.id = nextMedicineId++;
   printf("\n--- ADD NEW MEDICINE (ID: %d) ---\n", m.id);
   printf("Medicine Name: ");
   fgets(m.name, sizeof(m.name), stdin);
   m.name[strcspn(m.name, "\n")] = '\0';
   printf("Manufacturer: ");
   fgets(m.manufacturer, sizeof(m.manufacturer), stdin);
   m.manufacturer[strcspn(m.manufacturer, "\n")] = '\0';
```

```
    printf("Batch Number: ");
    fgets(m.batchNo, sizeof(m.batchNo), stdin);
    m.batchNo[strcspn(m.batchNo, "\n")] = '\0';
    printf("Category (0-Tablet 1-Syrup 2-Injection 3-Capsule 4-Ointment 5-Drops 6-Other):
");
    scanf("%d", &catChoice);
    if (catChoice < 0 || catChoice > 6) catChoice = 6;
    m.category = (Category)catChoice;
    printf("Quantity in Stock: ");
    scanf("%d", &m.quantity);
    printf("Unit Price (Rs.): ");
    scanf("%f", &m.unitPrice);
    clearInputBuffer();
    printf("Manufacturing Date (DD-MM-YYYY): ");
    fgets(m.mfgDate, sizeof(m.mfgDate), stdin);
    m.mfgDate[strcspn(m.mfgDate, "\n")] = '\0';

    printf("Expiry Date (DD-MM-YYYY): ");
    fgets(m.expiryDate, sizeof(m.expiryDate), stdin);
    m.expiryDate[strcspn(m.expiryDate, "\n")] = '\0';
    m.isActive = 1;
    inventory[medicineCount++] = m;
    saveMedicinesToFile();
    printf("\nMedicine '%s' added successfully with ID %d!\n", m.name, m.id);
}
/*
==========================================================================
==========
```

```
   INVENTORY: DISPLAY ALL
==========================================================================
========== */
void printMedicineRow(Medicine m) {
    printf("%-4d %-20.19s %-12.11s %-10.9s %-8d %-10.2f %-12s %-12s\n",
           m.id, m.name, categoryNames[m.category], m.batchNo,
           m.quantity, m.unitPrice, m.mfgDate, m.expiryDate);
}
void displayAllMedicines() {
    int found = 0;

printf("\n========================================================================
=====================\n");
    printf("%-4s %-20s %-12s %-10s %-8s %-10s %-12s %-12s\n",
           "ID", "Name", "Category", "Batch", "Qty", "Price", "MfgDate", "ExpDate");
printf("========================================================================
=====================\n");
    for (int i = 0; i < medicineCount; i++) {
        if (inventory[i].isActive) {
            printMedicineRow(inventory[i]);
            found = 1;
        }
    }
    if (!found) printf("No medicines found in inventory.\n");
printf("========================================================================
=====================\n");
    printf("Total active medicines: %d\n", medicineCount);
}
```

```
/*
================================================================
==========

  INVENTORY: SEARCH

================================================================
========== */

int findMedicineIndexById(int id) {
    for (int i = 0; i < medicineCount; i++) {
        if (inventory[i].id == id && inventory[i].isActive) return i;
    }
    return -1;
}

int findMedicineIndexByName(const char name[]) {
    char searchName[50], itemName[50];
    strcpy(searchName, name);
    toLowerStr(searchName);
    for (int i = 0; i < medicineCount; i++) {
        if (!inventory[i].isActive) continue;
        strcpy(itemName, inventory[i].name);
        toLowerStr(itemName);
        if (strstr(itemName, searchName) != NULL) return i;
    }
    return -1;
}

void searchMedicineMenu() {
    int choice;
    printf("\nSearch by: 1. ID   2. Name\nChoice: ");
    scanf("%d", &choice);
```

```
if (choice == 1) {
    int id;
    printf("Enter Medicine ID: ");
    scanf("%d", &id);
    int idx = findMedicineIndexById(id);
    if (idx == -1) {
        printf("\nMedicine with ID %d not found.\n", id);
    } else {
        printf("\n--- MEDICINE FOUND ---\n");
        printf("%-4s %-20s %-12s %-10s %-8s %-10s %-12s %-12s\n",
               "ID", "Name", "Category", "Batch", "Qty", "Price", "MfgDate", "ExpDate");
        printMedicineRow(inventory[idx]);
    }
} else if (choice == 2) {
    char name[50];
    clearInputBuffer();
    printf("Enter Medicine Name (or part of it): ");
    fgets(name, sizeof(name), stdin);
    name[strcspn(name, "\n")] = '\0';
    int idx = findMedicineIndexByName(name);
    if (idx == -1) {
        printf("\nNo medicine matching '%s' found.\n", name);
    } else {
        printf("\n--- MEDICINE FOUND ---\n");
        printf("%-4s %-20s %-12s %-10s %-8s %-10s %-12s %-12s\n",
               "ID", "Name", "Category", "Batch", "Qty", "Price", "MfgDate", "ExpDate");
        printMedicineRow(inventory[idx]);
```

```
        }
    } else {
        printf("\nInvalid choice.\n");
    }
}
/*
================================================================
==========
  INVENTORY: UPDATE STOCK
================================================================
========== */
void updateMedicineStock() {
    int id, qty, mode;
    printf("\nEnter Medicine ID to update: ");
    scanf("%d", &id);
    int idx = findMedicineIndexById(id);
    if (idx == -1) {
        printf("\nMedicine not found.\n");
        return;
    }
    printf("Current stock of '%s' = %d\n", inventory[idx].name, inventory[idx].quantity);
    printf("1. Add Stock   2. Reduce Stock   3. Set Exact Quantity\nChoice: ");
    scanf("%d", &mode);
    printf("Enter quantity: ");
    scanf("%d", &qty);
    switch (mode) {
        case 1: inventory[idx].quantity += qty; break;
        case 2:
```

```
            if (qty > inventory[idx].quantity) {
                printf("\nCannot reduce more than available stock!\n");
                return;
            }
            inventory[idx].quantity -= qty;
            break;
        case 3: inventory[idx].quantity = qty; break;
        default: printf("\nInvalid choice.\n"); return;
    }
    saveMedicinesToFile();
    printf("\nStock updated! New quantity of '%s' = %d\n", inventory[idx].name,
inventory[idx].quantity);
}
/*
================================================================
==========
  INVENTORY: DELETE (soft delete)
================================================================
========== */
void deleteMedicine() {
    int id;
    printf("\nEnter Medicine ID to delete: ");
    scanf("%d", &id);
    int idx = findMedicineIndexById(id);
    if (idx == -1) {
        printf("\nMedicine not found.\n");
        return;
    }
    char confirm;
```

```
    printf("Are you sure you want to delete '%s'? (y/n): ", inventory[idx].name);
    scanf(" %c", &confirm);
    if (tolower(confirm) == 'y') {
        inventory[idx].isActive = 0;
        saveMedicinesToFile();
        printf("\nMedicine '%s' deleted successfully.\n", inventory[idx].name);
    } else {
        printf("\nDeletion cancelled.\n");
    }
}

/*
================================================================
==========

  INVENTORY: LOW STOCK / EXPIRY ALERTS
================================================================
========== */

void checkLowStock() {
    int found = 0;
    printf("\n--- LOW STOCK ALERT (Threshold: %d) ---\n",
LOW_STOCK_THRESHOLD);
    printf("%-4s %-20s %-8s\n", "ID", "Name", "Qty");
    for (int i = 0; i < medicineCount; i++) {
        if (inventory[i].isActive && inventory[i].quantity < LOW_STOCK_THRESHOLD) {
            printf("%-4d %-20.19s %-8d\n", inventory[i].id, inventory[i].name,
inventory[i].quantity);
            found = 1;
        }
    }
    if (!found) printf("No medicines are low on stock.\n");
}
```

```
void checkExpiringMedicines() {
    /* Simplified check: compares expiry year/month string prefix entered by user */
    char cutoff[12];
    printf("\nEnter cutoff date to check expiry before (DD-MM-YYYY): ");
    clearInputBuffer();
    fgets(cutoff, sizeof(cutoff), stdin);
    cutoff[strcspn(cutoff, "\n")] = '\0';
    int found = 0;
    printf("\n--- MEDICINES EXPIRING BEFORE %s (string compare on YYYY-MM-DD basis) ---\n", cutoff);
    printf("Note: Enter dates consistently as DD-MM-YYYY for meaningful comparison.\n");
    printf("%-4s %-20s %-12s\n", "ID", "Name", "ExpDate");
    for (int i = 0; i < medicineCount; i++) {
        if (!inventory[i].isActive) continue;
        /* Reformat DD-MM-YYYY to YYYY-MM-DD for correct string comparison */
        char expReformat[12], cutReformat[12];
        sscanf(inventory[i].expiryDate, "%2s-%2s-%4s",
               &expReformat[6], &expReformat[3], expReformat);
        sscanf(cutoff, "%2s-%2s-%4s",
               &cutReformat[6], &cutReformat[3], cutReformat);
        expReformat[4] = '-'; expReformat[7] = '-';
        cutReformat[4] = '-'; cutReformat[7] = '-';
        expReformat[10] = '\0'; cutReformat[10] = '\0';
        if (strcmp(expReformat, cutReformat) < 0) {
            printf("%-4d %-20.19s %-12s\n", inventory[i].id, inventory[i].name,
inventory[i].expiryDate);
            found = 1;
        }
```

```
    }
    if (!found) printf("No medicines expiring before the given date.\n");
}
/*
================================================================
==========
  INVENTORY: SORTING (Bubble Sort)
================================================================
========== */
void sortMedicinesByName() {
    Medicine temp;
    for (int i = 0; i < medicineCount - 1; i++) {
        for (int j = 0; j < medicineCount - i - 1; j++) {
            if (strcasecmp(inventory[j].name, inventory[j + 1].name) > 0) {
                temp = inventory[j];
                inventory[j] = inventory[j + 1];
                inventory[j + 1] = temp;
            }
        }
    }
    printf("\nMedicines sorted by name.\n");
    displayAllMedicines();
    saveMedicinesToFile();
}
void sortMedicinesByQuantity() {
    Medicine temp;
    for (int i = 0; i < medicineCount - 1; i++) {
        for (int j = 0; j < medicineCount - i - 1; j++) {
```

```
            if (inventory[j].quantity > inventory[j + 1].quantity) {
                temp = inventory[j];
                inventory[j] = inventory[j + 1];
                inventory[j + 1] = temp;
            }
        }
    }
    printf("\nMedicines sorted by quantity (ascending).\n");
    displayAllMedicines();
    saveMedicinesToFile();
}
/*
==========================================================================
==========
   PRESCRIPTION: CREATE (also reduces medicine stock -> dispensing)
==========================================================================
========== */
void createPrescription() {
    if (prescriptionCount >= MAX_PRESCRIPTIONS) {
        printf("\nPrescription records full!\n");
        return;
    }
    Prescription p;
    p.prescriptionId = nextPrescriptionId++;
    p.itemCount = 0;
    p.totalAmount = 0.0f;
    clearInputBuffer();
    printf("\n--- CREATE PRESCRIPTION (ID: %d) ---\n", p.prescriptionId);
    printf("Patient Name: ");
```

```
fgets(p.patientName, sizeof(p.patientName), stdin);
p.patientName[strcspn(p.patientName, "\n")] = '\0';
printf("Patient Age: ");
scanf("%d", &p.patientAge);
clearInputBuffer();
printf("Doctor Name: ");
fgets(p.doctorName, sizeof(p.doctorName), stdin);
p.doctorName[strcspn(p.doctorName, "\n")] = '\0';
printf("Date (DD-MM-YYYY): ");
fgets(p.date, sizeof(p.date), stdin);
p.date[strcspn(p.date, "\n")] = '\0';
int addMore = 1;
while (addMore && p.itemCount < MAX_ITEMS_PER_PRESCRIPTION) {
    int id, qty;
    printf("\nEnter Medicine ID to prescribe: ");
    scanf("%d", &id);
    int idx = findMedicineIndexById(id);
    if (idx == -1) {
        printf("Medicine not found. Try again.\n");
        continue;
    }

    printf("Available stock: %d. Enter quantity to prescribe: ", inventory[idx].quantity);
    scanf("%d", &qty);
    if (qty <= 0 || qty > inventory[idx].quantity) {
        printf("Invalid quantity or insufficient stock!\n");
        continue;
```

```
    }
    /* Dispense: reduce stock */
    inventory[idx].quantity -= qty;
    PrescribedItem item;
    item.medicineId = inventory[idx].id;
    strcpy(item.medicineName, inventory[idx].name);
    item.quantity = qty;
    item.subTotal = qty * inventory[idx].unitPrice;
    p.items[p.itemCount++] = item;
    p.totalAmount += item.subTotal;
    printf("Added %d x %s to prescription. Subtotal: Rs.%.2f\n",
           qty, item.medicineName, item.subTotal);
    clearInputBuffer();
    printf("Add another medicine? (y/n): ");
    char c;
    scanf(" %c", &c);
    addMore = (tolower(c) == 'y');
}
if (p.itemCount == 0) {
    printf("\nNo medicines added. Prescription cancelled.\n");
    nextPrescriptionId--;
    return;
}
prescriptions[prescriptionCount++] = p;
saveMedicinesToFile();
savePrescriptionsToFile();
printf("\n--- PRESCRIPTION CREATED SUCCESSFULLY ---\n");
```

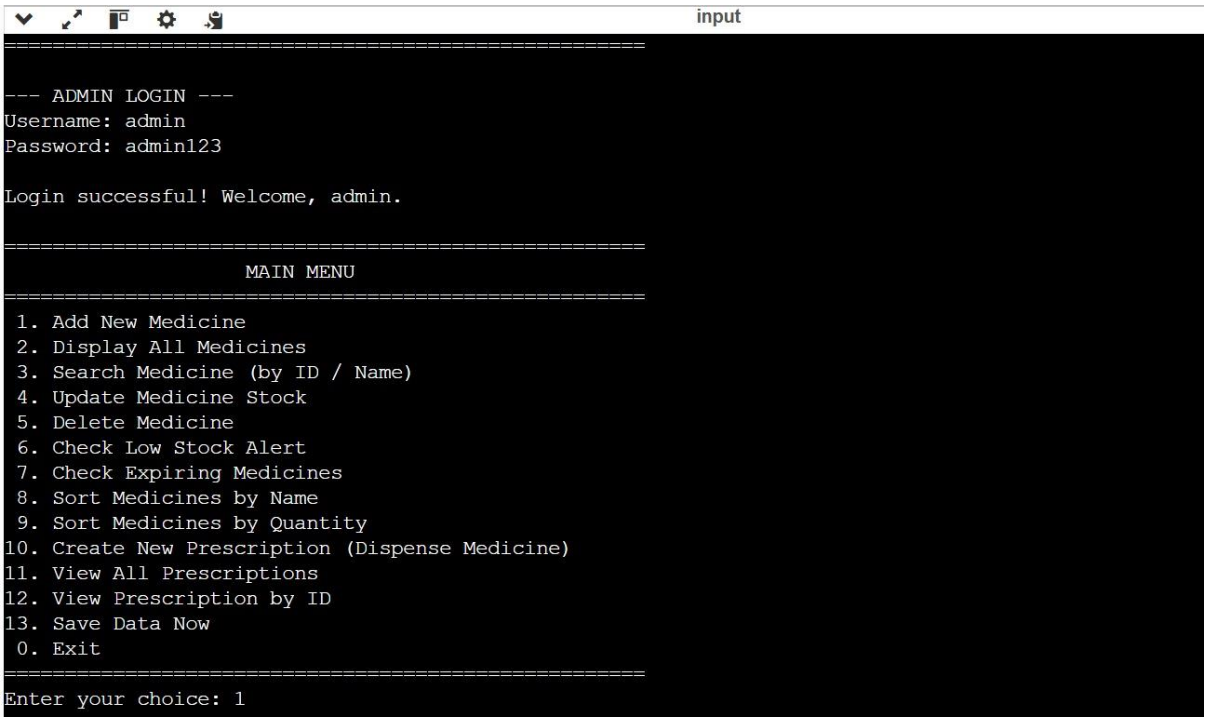
```
    printPrescription(p);
}

/*
================================================================
==========

  PRESCRIPTION: VIEW

================================================================
========== */

void printPrescription(Prescription p) {
printf("\n============================================================\n")
;
    printf("Prescription ID : %d\n", p.prescriptionId);
    printf("Patient         : %s (Age: %d)\n", p.patientName, p.patientAge);
    printf("Doctor          : %s\n", p.doctorName);
    printf("Date            : %s\n", p.date);
    printf("------------------------------------------------------------\n");
    printf("%-6s %-20s %-6s %-10s\n", "MedID", "Medicine", "Qty", "SubTotal");
    for (int i = 0; i < p.itemCount; i++) {
        printf("%-6d %-20.19s %-6d Rs.%-8.2f\n",
               p.items[i].medicineId, p.items[i].medicineName,
               p.items[i].quantity, p.items[i].subTotal);
    }
    printf("------------------------------------------------------------\n");
    printf("TOTAL AMOUNT    : Rs.%.2f\n", p.totalAmount);
printf("============================================================\n");
}
void viewAllPrescriptions() {
    if (prescriptionCount == 0) {
        printf("\nNo prescriptions recorded yet.\n");
        return;
```

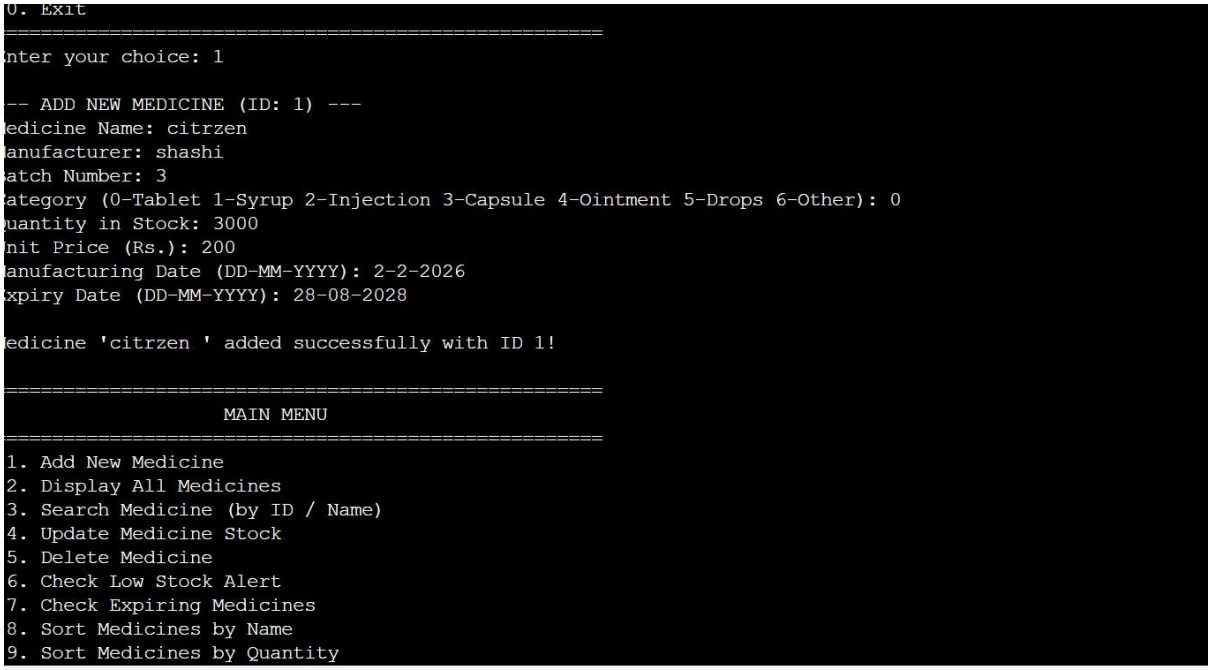
```
    }
    for (int i = 0; i < prescriptionCount; i++) {
        printPrescription(prescriptions[i]);
    }
}
void viewPrescriptionById() {
    int id;
    printf("\nEnter Prescription ID: ");
    scanf("%d", &id);
    for (int i = 0; i < prescriptionCount; i++) {
        if (prescriptions[i].prescriptionId == id) {
            printPrescription(prescriptions[i]);
            return;
        }
    }
    printf("\nPrescription with ID %d not found.\n", id);
}
/*
================================================================
==========

  FILE HANDLING (Binary persistence)
================================================================
========== */
void saveMedicinesToFile() {
    FILE *fp = fopen(MEDICINE_FILE, "wb");
    if (fp == NULL) {
        printf("\nError: Could not save medicine data.\n");
        return;
    }
```

```
    fwrite(&medicineCount, sizeof(int), 1, fp);
    fwrite(&nextMedicineId, sizeof(int), 1, fp);
    fwrite(inventory, sizeof(Medicine), medicineCount, fp);
    fclose(fp);
}
void loadMedicinesFromFile() {
    FILE *fp = fopen(MEDICINE_FILE, "rb");
    if (fp == NULL) return; /* No existing data yet */
    fread(&medicineCount, sizeof(int), 1, fp);
    fread(&nextMedicineId, sizeof(int), 1, fp);
    fread(inventory, sizeof(Medicine), medicineCount, fp);
    fclose(fp);
}
void savePrescriptionsToFile() {
    FILE *fp = fopen(PRESCRIPTION_FILE, "wb");
    if (fp == NULL) {
        printf("\nError: Could not save prescription data.\n");
        return;
    }
    fwrite(&prescriptionCount, sizeof(int), 1, fp);
    fwrite(&nextPrescriptionId, sizeof(int), 1, fp);
    fwrite(prescriptions, sizeof(Prescription), prescriptionCount, fp);
    fclose(fp);
}
void loadPrescriptionsFromFile() {
    FILE *fp = fopen(PRESCRIPTION_FILE, "rb");
    if (fp == NULL) return;
```

```
    fread(&prescriptionCount, sizeof(int), 1, fp);
    fread(&nextPrescriptionId, sizeof(int), 1, fp);
    fread(prescriptions, sizeof(Prescription), prescriptionCount, fp);
    fclose(fp);
}
/*
================================================================
==========
  UTILITY FUNCTIONS
================================================================
========== */
void clearInputBuffer() {
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}
void toLowerStr(char *str) {
    for (int i = 0; str[i]; i++) {
        str[i] = tolower((unsigned char)str[i]);
    }
}
void pause_screen() {
    printf("\nPress Enter to continue...");
    clearInputBuffer();
    getchar();
}
```

Project Outcome





```
9. Sort Medicines by Quantity
0. Create New Prescription (Dispense Medicine)
1. View All Prescriptions
2. View Prescription by ID
3. Save Data Now
0. Exit
==================================================
nter your choice: 1

-- ADD NEW MEDICINE (ID: 2) ---
edicine Name: pp
anufacturer: kl
atch Number: 3
ategory (0-Tablet 1-Syrup 2-Injection 3-Capsule 4-Ointment 5-Drops 6-Other): 4
uantity in Stock: 299
nit Price (Rs.): 26
anufacturing Date (DD-MM-YYYY): 12-09-1999
xpiry Date (DD-MM-YYYY): 20-09-2028

edicine 'pp' added successfully with ID 2!

==================================================
                  MAIN MENU
==================================================
1. Add New Medicine
```